

Лекция по JavaScript (Часть 5)

Продолжительность: 3 часа

Предыдущие темы:

1. Введение в JavaScript: синтаксис, переменные, типы данных, операторы.
 2. Управляющие конструкции: условия, циклы, функции.
 3. Массивы, объекты, работа с DOM, асинхронность.
 4. Модули, замыкания, классы и ООП.
-

План лекции:

1. **Работа с API: `fetch`, `axios` (1 час)**
 - Что такое API?
 - Использование `fetch` для запросов.
 - Использование `axios` для запросов.
 - Практические примеры.
 2. **Обработка ошибок в асинхронном коде (45 минут)**
 - Обработка ошибок в `fetch`.
 - Обработка ошибок в `axios`.
 - Практические примеры.
 3. **Продвинутые темы: итераторы, генераторы, прокси (1 час)**
 - Итераторы и итерируемые объекты.
 - Генераторы и их применение.
 - Прокси и их использование.
 - Практические примеры.
 4. **Перерыв (10 минут)**
 5. **Практика и закрепление материала (1 час)**
 - Решение задач на работу с API, обработку ошибок и продвинутые темы.
 - Ответы на вопросы.
-

Подробное содержание лекции с комментариями:

1. Работа с API: `fetch`, `axios` (1 час)

Теория:

API (Application Programming Interface):

- Это набор правил и протоколов, которые позволяют программам взаимодействовать друг с другом.
- В веб-разработке API часто используются для получения данных с сервера.

`fetch`:

- Встроенный метод для выполнения HTTP-запросов.
- Возвращает промис, который разрешается в ответ сервера.

`axios`:

- Популярная библиотека для выполнения HTTP-запросов.
- Упрощает работу с запросами и ответами.

Пример 1: Использование `fetch`

```
// Выполняем GET-запрос к API
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json(); // Преобразуем ответ в JSON
  })
  .then(data => {
    console.log(data); // Выводим данные
  })
  .catch(error => {
    console.error('There was a problem with the fetch operation:',
error);
  });
```

Комментарии:

- `fetch` возвращает промис, который разрешается в объект `Response`.
- Метод `response.json()` преобразует ответ в JSON.
- Обработка ошибок выполняется с помощью `.catch`.

Пример 2: Использование `axios`

```
// Устанавливаем axios (если не установлен)
// npm install axios

import axios from 'axios';

// Выполняем GET-запрос к API
axios.get('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    console.log(response.data); // Выводим данные
  })
  .catch(error => {
    console.error('There was a problem with the axios request:', error);
  });
```

Комментарии:

- `axios` автоматически преобразует ответ в JSON.
- Обработка ошибок выполняется с помощью `.catch`.

Практическое задание:

1. Используйте `fetch` для получения списка пользователей с `https://jsonplaceholder.typicode.com/users`.
2. Используйте `axios` для получения списка постов с `https://jsonplaceholder.typicode.com/posts`.

Пример решения:

```
// fetch
fetch('https://jsonplaceholder.typicode.com/users')
  .then(response => response.json())
  .then(users => {
    console.log(users);
  })
  .catch(error => {
    console.error('Error fetching users:', error);
  });

// axios
axios.get('https://jsonplaceholder.typicode.com/posts')
  .then(response => {
    console.log(response.data);
  });
```

```
    })  
    .catch(error => {  
        console.error('Error fetching posts:', error);  
    });
```

2. Обработка ошибок в асинхронном коде (45 минут)

Теория:

Обработка ошибок в асинхронном коде важна для предотвращения сбоев и улучшения пользовательского опыта.

Пример 1: Обработка ошибок в `fetch`

```
fetch('https://jsonplaceholder.typicode.com/invalid-url')  
  .then(response => {  
    if (!response.ok) {  
      throw new Error('Network response was not ok');  
    }  
    return response.json();  
  })  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error('Fetch error:', error);  
  });
```

Комментарии:

- Если URL неверный или сервер возвращает ошибку, выбрасывается исключение.
- Ошибка перехватывается в блоке `.catch`.

Пример 2: Обработка ошибок в `axios`

```
axios.get('https://jsonplaceholder.typicode.com/invalid-url')  
  .then(response => {  
    console.log(response.data);  
  })  
  .catch(error => {  
    if (error.response) {
```

```

        // Ошибка сервера
        console.error('Server error:', error.response.status);
    } else if (error.request) {
        // Ошибка сети
        console.error('Network error:', error.request);
    } else {
        // Другие ошибки
        console.error('Error:', error.message);
    }
});

```

Комментарии:

- `axios` предоставляет подробную информацию об ошибках.
- Ошибки можно разделить на серверные, сетевые и другие.

Практическое задание:

1. Используйте `fetch` для получения данных с неверного URL и обработайте ошибку.
2. Используйте `axios` для получения данных с неверного URL и обработайте ошибку.

Пример решения:

```

// fetch
fetch('https://jsonplaceholder.typicode.com/invalid-url')
    .then(response => {
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        return response.json();
    })
    .then(data => {
        console.log(data);
    })
    .catch(error => {
        console.error('Fetch error:', error);
    });

// axios
axios.get('https://jsonplaceholder.typicode.com/invalid-url')
    .then(response => {
        console.log(response.data);
    })
    .catch(error => {

```

```
    console.error('Axios error:', error);
  });
```

3. Продвинутые темы: итераторы, генераторы, прокси (1 час)

Конечно! Давайте подробно разберём, зачем нужны итераторы, генераторы и прокси, где они используются на практике, и приведём примеры их применения.

1. Итераторы

Зачем нужны:

Итераторы позволяют перебирать элементы коллекции (например, массивы, строки, объекты) в определённом порядке. Они предоставляют унифицированный способ работы с разными типами данных, которые можно итерировать.

Где используются:

- **Перебор элементов массива:** Встроенные методы массива (`forEach` , `map` , `filter`) используют итераторы.
- **Работа с коллекциями:** Итераторы используются в `Set` , `Map` и других структурах данных.
- **Создание пользовательских итерируемых объектов:** Например, для работы с данными, которые поступают по частям (пагинация, потоковые данные).

Пример 1: Встроенные итераторы

```
const arr = [1, 2, 3];
// Получаем итератор массива с помощью метода [Symbol.iterator]
const iterator = arr[Symbol.iterator]();

// Используем метод next() для перебора элементов
console.log(iterator.next()); // { value: 1, done: false } – первый элемент
console.log(iterator.next()); // { value: 2, done: false } – второй элемент
console.log(iterator.next()); // { value: 3, done: false } – третий элемент
console.log(iterator.next()); // { value: undefined, done: true } – итерация завершена
```

Комментарии:

- `[Symbol.iterator]` — это специальный метод, который возвращает итератор для объекта.
- Метод `next()` возвращает объект с двумя свойствами:
 - `value` — текущее значение.
 - `done` — флаг, указывающий, завершена ли итерация.

Пример 2: Пользовательский итератор

```
const range = {
  from: 1, // Начальное значение
  to: 5,   // Конечное значение

  // Метод [Symbol.iterator] возвращает итератор
  [Symbol.iterator]() {
    let current = this.from; // Текущее значение
    return {
      // Метод next() возвращает следующее значение
      next: () => {
        if (current <= this.to) {
          // Возвращаем текущее значение и увеличиваем его на 1
          return { value: current++, done: false };
        } else {
          // Итерация завершена
          return { done: true };
        }
      }
    };
  }
};

// Используем for...of для перебора итератора
for (const num of range) {
  console.log(num); // 1, 2, 3, 4, 5
}
```

Комментарии:

- Мы создали объект `range`, который можно итерировать.
- Метод `[Symbol.iterator]` возвращает объект с методом `next()`, который управляет итерацией.
- `for...of` автоматически вызывает метод `next()` и перебирает значения,

пока `done` не станет `true`.

Практическое применение:

- Пагинация данных (постраничная загрузка).
 - Работа с бесконечными последовательностями (например, генерация уникальных ID).
-

2. Генераторы

Зачем нужны:

Генераторы позволяют создавать функции, которые могут приостанавливать своё выполнение и возвращать несколько значений. Это полезно для работы с асинхронными операциями, бесконечными последовательностями или сложными итерируемыми объектами.

Где используются:

- **Ленивые вычисления:** Генераторы позволяют вычислять значения по мере необходимости.
- **Асинхронные операции:** Генераторы используются вместе с `async/await` для упрощения работы с асинхронным кодом.
- **Генерация последовательностей:** Например, числа Фибоначчи, случайные числа, уникальные идентификаторы.

Пример 1: Ленивые вычисления

```
// Создаём генератор с помощью function*
function* generateNumbers() {
  yield 1; // Возвращаем 1 и приостанавливаем выполнение
  yield 2; // Возвращаем 2 и приостанавливаем выполнение
  yield 3; // Возвращаем 3 и приостанавливаем выполнение
}

// Создаём объект генератора
const gen = generateNumbers();

// Используем метод next() для получения значений
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
```



```
console.log(gen.next().value); // 3
console.log(gen.next().value); // undefined (генератор завершён)
```

Комментарии:

- Генератор создаётся с помощью `function*`.
- Ключевое слово `yield` приостанавливает выполнение функции и возвращает значение.
- Метод `next()` возобновляет выполнение генератора до следующего `yield`.

Пример 2: Бесконечная последовательность

```
// Генератор, который возвращает бесконечную последовательность чисел
function* infiniteSequence() {
  let i = 0;
  while (true) {
    yield i++; // Возвращаем текущее значение и увеличиваем его на 1
  }
}

// Создаём объект генератора
const gen = infiniteSequence();

// Получаем значения по одному
console.log(gen.next().value); // 0
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
```

Комментарии:

- Генератор может работать бесконечно, так как `while (true)` никогда не завершается.
- Каждый вызов `next()` возвращает следующее значение.

Пример 3: Асинхронный генератор

```
// Асинхронный генератор, который получает данные по URL
async function* fetchData(urls) {
  for (const url of urls) {
    // Выполняем асинхронный запрос
    const response = await fetch(url);
    // Преобразуем ответ в JSON
    const data = await response.json();
  }
}
```

```
        // Возвращаем данные
        yield data;
    }
}

// Используем асинхронный генератор
(async () => {
    const urls = [
        "https://jsonplaceholder.typicode.com/posts/1",
        "https://jsonplaceholder.typicode.com/posts/2"
    ];

    // Перебираем данные с помощью for await...of
    for await (const data of fetchData(urls)) {
        console.log(data); // Выводим данные каждого поста
    }
})();
```

Комментарии:

- Асинхронный генератор позволяет работать с асинхронными операциями (например, запросами к API).
- `for await...of` используется для перебора асинхронных данных.

Практическое применение:

- Ленивая загрузка данных (например, постраничная загрузка).
- Генерация уникальных ID или случайных чисел.
- Упрощение асинхронного кода.

3. Прокси

Зачем нужны:

Прокси позволяют перехватывать и изменять поведение объектов. Это мощный инструмент для создания гибких и расширяемых систем, таких как валидация данных, логирование, кэширование и т.д.

Где используются:

- **Валидация данных:** Прокси могут проверять значения перед их установкой.
- **Логирование:** Прокси могут записывать все операции с объектом.

- **Кэширование:** Прокси могут кэшировать результаты вызовов методов.
- **Защита данных:** Прокси могут ограничивать доступ к определённым свойствам объекта.

Пример 1: Логирование

```
const target = {
  name: "John",
  age: 30
};

// Обработчик для прокси
const handler = {
  // Перехватываем операцию чтения свойства
  get(target, prop) {
    console.log(`Getting property: ${prop}`);
    return target[prop]; // Возвращаем значение свойства
  },
  // Перехватываем операцию записи свойства
  set(target, prop, value) {
    console.log(`Setting property: ${prop} to ${value}`);
    target[prop] = value; // Устанавливаем значение свойства
    return true; // Указываем, что операция успешна
  }
};

// Создаём прокси
const proxy = new Proxy(target, handler);

// Используем прокси
console.log(proxy.name); // Getting property: name, John
proxy.age = 31; // Setting property: age to 31
```

Комментарии:

- Прокси перехватывает операции чтения (`get`) и записи (`set`) свойств.
- Мы можем добавить дополнительную логику (например, логирование) перед выполнением операции.

Пример 2: Валидация данных

```
const user = {
  name: "John",
  age: 30
```

```

};

// Обработчик для прокси
const handler = {
  // Перехватываем операцию записи свойства
  set(target, prop, value) {
    if (prop === 'age' && typeof value !== 'number') {
      throw new TypeError("Age must be a number"); // Проверяем тип
значения
    }
    target[prop] = value; // Устанавливаем значение свойства
    return true; // Указываем, что операция успешна
  }
};

// Создаём прокси
const proxy = new Proxy(user, handler);

proxy.age = 31; // Работает
proxy.age = "31"; // Ошибка: Age must be a number

```

Комментарии:

- Прокси позволяет проверять значения перед их установкой.
- Если значение не проходит проверку, выбрасывается ошибка.

Пример 3: Кэширование

```

const cache = new Map(); // Хранилище для кэша

// Обработчик для прокси
const handler = {
  // Перехватываем операцию чтения свойства
  get(target, prop) {
    if (cache.has(prop)) {
      console.log("Returning cached result");
      return cache.get(prop); // Возвращаем значение из кэша
    }
    const result = target[prop]; // Вычисляем значение
    cache.set(prop, result); // Сохраняем значение в кэше
    return result; // Возвращаем значение
  }
};

// Создаём прокси

```

```
const math = new Proxy({
  square(x) {
    return x * x; // Метод для вычисления квадрата числа
  }
}, handler);

console.log(math.square(5)); // 25 (вычислено)
console.log(math.square(5)); // 25 (возвращено из кэша)
```

Комментарии:

- Прокси кэширует результаты вызовов методов.
- При повторном вызове метода результат возвращается из кэша, что ускоряет выполнение.

Практическое применение:

- Логирование всех операций с объектом.
 - Валидация данных перед их сохранением.
 - Кэширование результатов вызовов методов.
 - Создание защищённых объектов (например, с ограниченным доступом к свойствам).
-

Итог:

- **Итераторы:** Универсальный способ перебора элементов коллекций. Используются для работы с массивами, строками, объектами и пользовательскими структурами данных.
 - **Генераторы:** Позволяют создавать функции, которые могут приостанавливать своё выполнение и возвращать несколько значений. Используются для ленивых вычислений, асинхронного кода и генерации последовательностей.
 - **Прокси:** Позволяют перехватывать и изменять поведение объектов. Используются для валидации, логирования, кэширования и защиты данных.
-

Домашнее задание:

1. Создайте итератор, который возвращает квадраты чисел от 1 до 10.
2. Напишите генератор, который возвращает случайные числа в заданном диапазоне.

3. Используйте прокси для создания объекта, который запрещает изменение определённых свойств.

Пример решения:

```
// Итератор (квадраты чисел)
const squares = {
  [Symbol.iterator]() {
    let current = 1;
    return {
      next() {
        if (current <= 10) {
          return { value: current * current++, done: false };
        }
        return { done: true };
      }
    };
  }
};

for (const square of squares) {
  console.log(square); // 1, 4, 9, 16, ..., 100
}

// Генератор (случайные числа)
function* randomNumbers(min, max) {
  while (true) {
    yield Math.floor(Math.random() * (max - min + 1)) + min;
  }
}

const randomGen = randomNumbers(1, 10);
console.log(randomGen.next().value); // Случайное число от 1 до 10

// Прокси (запрет изменения свойств)
const user = {
  name: "John",
  age: 30
};

const handler = {
  set(target, prop, value) {
    if (prop === 'name') {
      throw new Error("Name cannot be changed");
    }
    target[prop] = value;
  }
};
```

```
        return true;
    }
};

const proxy = new Proxy(user, handler);

proxy.age = 31; // Работает
proxy.name = "Alice"; // Ошибка: Name cannot be changed
```

Итог:

- `fetch` и `axios` — основные инструменты для работы с API.
 - **Обработка ошибок** важна для стабильности приложения.
 - **Итераторы, генераторы и прокси** — мощные инструменты для работы с данными и объектами.
-

Следующая лекция:

- Работа с файлами и формами.
- Веб-сокеты и реальное время.
- Оптимизация производительности в JavaScript.